



MATEUS MENDES DA SILVA

**ANÁLISE COMPARATIVA DE ESTRATÉGIAS DE
MIGRAÇÃO DE ARQUITETURAS MONOLÍTICAS PARA
MICROSSERVIÇOS**

LAVRAS – MG

2026

MATEUS MENDES DA SILVA

**ANÁLISE COMPARATIVA DE ESTRATÉGIAS DE MIGRAÇÃO DE
ARQUITETURAS MONOLÍTICAS PARA MICROSERVIÇOS**

Monografia apresentada à Universidade Federal de Lavras, como parte das exigências do Curso de Ciência da Computação, para a obtenção do título de Bacharel.

Prof. DSc. Heitor Augustus Xavier Costa
Orientador

**LAVRAS – MG
2026**

**Ficha catalográfica elaborada pela Coordenadoria de Processos Técnicos da Biblioteca
Universitária da UFLA, com dados informados pelo(a) próprio(a) autor(a).**

Silva, Joana Autora da

Análise Comparativa de Estratégias de Migração de Arquiteturas Monolíticas para Microserviços / Joana Autora da Silva, João Autor do Silvo. 6^a ed. rev., atual. e ampl. – Lavras : UFLA, 2026.

26p. : il.

Tese(doutorado)–Universidade Federal de Lavras, 2026.

Orientador: Prof. DSc. Heitor Augustus Xavier Costa.

Bibliografia.

1. TCC. 2. Monografia. 3. Dissertação. 4. Tese. 5. Trabalho Científico – Normas. I. Universidade Federal de Lavras. II. Título.

CDD 808.066

MATEUS MENDES DA SILVA

**ANÁLISE COMPARATIVA DE ESTRATÉGIAS DE MIGRAÇÃO DE
ARQUITETURAS MONOLÍTICAS PARA MICROSERVIÇOS**

Monografia apresentada à Universidade Federal de Lavras, como parte das exigências do Curso de Ciência da Computação, para a obtenção do título de Bacharel.

APROVADA em 29 de maio de 2025.

Prof. Me. Antônio Banca Um	UFM
Prof. Dr. João Banca Dois	FCO
Profa. Dra. Eliza Banca Três	BELMIS
Prof. Esp. Zorbison Uberplax IV	UFO

Prof. DSc. Heitor Augustus Xavier Costa
Orientador

**LAVRAS – MG
2026**

Espaço reservado à dedicatória.

AGRADECIMENTOS

Espaço reservado aos agradecimentos.

Espaço reservado à epígrafe.

(Autor Desconhecido)

RESUMO

O crescimento acelerado da indústria de software e a necessidade de escalabilidade em sistemas legados intensificam a busca por estratégias eficientes de modernização arquitetural. Diante desse cenário, a transição de arquiteturas monolíticas para microsserviços surge como alternativa para reduzir o débito técnico e aumentar a agilidade de entrega, embora os riscos operacionais desse processo demandem avaliações empíricas rigorosas. Este trabalho analisa de forma comparativa e aplicada as estratégias *Strangler Fig* e *Branch by Abstraction*, integradas no contexto de um sistema de porte médio utilizando o ecossistema Java e *Spring Boot*.

Palavras-chave: Engenharia de Software; Microsserviços; Arquitetura Monolítica; Migração de Sistemas; *Strangler Fig*; *Branch by Abstraction*; *Spring Boot*.

ABSTRACT

The abstract must contain words representative of the content of the work, located below it, preceded by the expression “keywords” followed by two points. Keywords should be separated from each other by semicolons and finalized by periods. They should be spelled with the initials in lowercase, with the exception of proper nouns and scientific names.

Keywords: summary; words; representative.

LISTA DE FIGURAS

LISTA DE TABELAS

SUMÁRIO

1	INTRODUÇÃO	11
1.1	Motivação	11
1.2	Objetivo	12
1.3	Estrutura do Trabalho	12
2	REFERENCIAL TEÓRICO	14
2.1	Arquitetura Monolítica	14
2.2	Arquitetura de Microsserviços	14
2.3	Estratégias de Migração de Monólitos para Microsserviços	15
2.3.1	Padrão do Figo Estrangulador (<i>Strangler Fig Pattern</i>)	15
2.3.2	Ramificação por Abstração (<i>Branch by Abstraction</i>)	15
2.4	Tecnologias Habilitadoras	16
2.4.1	Linguagem de Programação Java	16
2.4.2	Plataforma Docker e Contêinerização	16
2.4.3	Protocolo HTTP para Comunicação Distribuída	17
3	METODOLOGIA	18
4	DESENVOLVIMENTO	19
5	DISCUSSÃO DOS RESULTADOS	20
6	CONCLUSÃO	21
	REFERÊNCIAS	22
	ANEXOS	24
	APÊNDICES	26

1 INTRODUÇÃO

O desenvolvimento de sistemas de software tem passado por transformações profundas para atender às demandas de escalabilidade e rápida entrega de funcionalidades. Historicamente, a abordagem predominante tem sido a arquitetura monolítica, na qual todos os componentes de um sistema são desenvolvidos, implantados e escalados como uma unidade única. Embora essa estrutura simplifique o desenvolvimento inicial, ela apresenta limitações críticas à medida que o sistema cresce, resultando em códigos acoplados, dificuldades de manutenção e obstáculos para a implementação de novas tecnologias (Newman, 2020).

Nesse cenário, surge a necessidade de modelos mais flexíveis. Segundo (Tapia *et al.*, 2020), a transição de sistemas monolíticos para microsserviços tornou-se uma tendência significativa na engenharia de software moderna, motivada pela busca por melhor desempenho e eficiência operacional. A arquitetura de microsserviços propõe a decomposição da aplicação em um conjunto de serviços pequenos e independentes, que se comunicam através de protocolos leves. Essa modularização permite que cada componente seja desenvolvido e escalado de forma isolada, conferindo maior agilidade às equipes de desenvolvimento (Newman, 2020).

Entretanto, a migração de um monólito para uma arquitetura distribuída como os microsserviços não é uma tarefa trivial e envolve uma série de decisões estratégicas que, antes de tudo, devem ser bem planejadas para que essa transição ocorra de forma eficiente e segura. É fundamental analisar o sistema como um todo e identificar os pontos críticos que podem impactar a transição, além de considerar os impactos sobre a infraestrutura e os processos existentes. A mudança exige não apenas uma reestruturação técnica, mas também uma abordagem evolutiva, focada em padrões que minimizem os riscos de interrupção dos serviços existentes.

1.1 Motivação

Com o crescimento exponencial dos sistemas de software e a recente evolução tecnológica, a arquitetura monolítica tem se mostrado cada vez mais inadequada para atender às demandas de escalabilidade e agilidade. Softwares cada vez mais complexos, que dificultam a manutenção e a sua evolução, tornando a arquitetura monolítica obsoleta e de difícil manutenção. A simplicidade inicial do monólito se torna um obstáculo à medida que o sistema cresce,

resultando em códigos acoplados, dificuldades de manutenção e obstáculos para a implementação de novas tecnologias.

Nesse cenário, pequenas alterações em uma parte do código podem gerar impactos imprevistos em todo o sistema, tornando os ciclos de teste e implantação excessivamente lentos e arriscados. A dificuldade de escalar componentes específicos de forma independente também resulta em um uso ineficiente de recursos computacionais, uma vez que é necessário replicar a aplicação inteira para atender à demanda de apenas uma de suas funções.

Nesse contexto, a arquitetura de microsserviços surge como uma solução promissora para superar as limitações do monólito, oferecendo uma abordagem modular e flexível que permite o desenvolvimento e a implantação de serviços independentes. A migração para microsserviços, embora desafiadora, é motivada pela necessidade de melhorar a escalabilidade, a agilidade e a eficiência operacional dos sistemas de software modernos.

1.2 Objetivo

Diante do cenário apresentado, é visível que a arquitetura monolítica tem se tornado cada vez mais inadequada para atender às demandas de escalabilidade e agilidade dos sistemas de software modernos, e, assim, surge a necessidade da migração de serviços monolíticos já consolidados e funcionais para uma arquitetura mais moderna e flexível, como a arquitetura de microsserviços, de forma estratégica e segura.

Este trabalho tem como objetivo comparar duas diferentes abordagens para a migração de arquiteturas monolíticas para microsserviços, sendo elas a *Strangler Fig* e *Branch by Abstraction*. Por meio da análise de boas práticas e estratégias de decomposição gradual, o estudo busca fornecer um referencial técnico e gerencial para equipes de desenvolvimento que enfrentam o desafio de modernizar sistemas legados, assegurando uma transição que potencialize a escalabilidade do software e a autonomia organizacional.

1.3 Estrutura do Trabalho

Este trabalho está organizado em X(a definir) capítulos, incluindo esta introdução. O Capítulo 2 apresenta a fundamentação teórica, explorando as características das arquiteturas monolíticas e de microsserviços, bem como os padrões evolutivos essenciais para a moderniza-

ção de sistemas. No Capítulo 3, detalha-se o percurso metodológico adotado, descrevendo os critérios de análise e as ferramentas utilizadas para a condução do estudo. O Capítulo 4 é dedicado ao desenvolvimento do trabalho, no qual se descreve a aplicação prática das estratégias de decomposição gradual do sistema legado. Na sequência, o Capítulo 5 apresenta a análise e discussão dos resultados obtidos, avaliando os impactos da transição na escalabilidade técnica e em métricas quantitativas. Por fim, o Capítulo 6 sintetiza as conclusões da análise comparativa realizada, destacando as principais contribuições do estudo e sugerindo caminhos para pesquisas futuras.

2 REFERENCIAL TEÓRICO

2.1 Arquitetura Monolítica

A arquitetura monolítica é historicamente descrita como o modelo tradicional de desenvolvimento de software em que todas as capacidades de um sistema estão concentradas e integradas em uma unidade única de implantação (deployment unit) (Newman, 2020). Segundo Clemente e Silva (2020), esse ecossistema costuma ser composto por um único bloco modular centralizado que realiza toda a comunicação com a base de dados. Nos estágios iniciais de um projeto, essa centralização oferece vantagens práticas evidentes, como fluxos de desenvolvimento diretos, facilidade na execução de testes de integração e deploys simplificados pelo envio de um único artefato consolidado.

Entretanto, à medida que a aplicação expande seu ciclo de vida, recebe atualizações e incorpora novas equipes, as limitações intrínsecas ao monólito tornam-se nítidas. Conforme apontado por (Tapia *et al.*, 2020), o aumento descontrolado da base de código gera um acoplamento modular severo e complexo, elevando o débito técnico e prejudicando de forma visível a produtividade dos times de desenvolvimento na manutenção do sistema. Sob a ótica da infraestrutura, o monólito impõe uma escalabilidade ineficiente e onerosa, visto que o aumento de demanda em uma funcionalidade isolada obriga a organização a replicar horizontalmente a aplicação inteira, gerando desperdício de recursos computacionais (Newman, 2020).

2.2 Arquitetura de Microsserviços

Em contraposição à rigidez das aplicações centralizadas, a arquitetura de microsserviços organiza o sistema como uma suíte de serviços modulares de escopo reduzido, independentes e altamente autônomos (Newman, 2020). Cada microsserviço é projetado em torno de um domínio ou capacidade de negócio específica, executando seu próprio processo e realizando a comunicação com os demais componentes do ecossistema por meio de protocolos de rede leves e padronizados, tipicamente APIs baseadas em HTTP ou sistemas de mensageria (Tapia *et al.*, 2020).

Dentre os principais benefícios técnicos desse estilo arquitetural, destaca-se a capacidade de implantação independente (independent deployability), que permite que um serviço

seja modificado, testado e publicado sem a necessidade de coordenar ou reimplantar o restante da plataforma (Newman, 2020). Além disso, o isolamento de falhas atua como uma barreira de robustez técnica: a degradação ou queda de um microsserviço sob estresse não gera um efeito cascata que comprometa a aplicação por completo, preservando a disponibilidade das outras funções de negócio em execução (Tapia *et al.*, 2020).

2.3 Estratégias de Migração de Monólitos para Microsserviços

2.3.1 Padrão do Figo Estrangulador (*Strangler Fig Pattern*)

O padrão do Figo Estrangulador (*Strangler Fig Pattern*) é consolidado na literatura como uma das principais abordagens evolutivas para a modernização de sistemas legados, inspirando-se na metáfora da planta que cresce ao redor de uma árvore até substituí-la (NEWMAN, 2020). Na engenharia de software, essa estratégia baseia-se em interceptar de maneira incremental as requisições direcionadas ao perímetro do monólito, frequentemente com o suporte de *proxies* de roteamento ou *API Gateways*, e encaminhá-las para os novos microsserviços à medida que suas funcionalidades correspondentes são implementadas (CLEMENTE; SILVA, 2020). Como detalha Newman (2020), o grande trunfo desse padrão é permitir que a arquitetura legada e a nova estrutura distribuída coexistam de forma segura em ambiente produtivo, garantindo uma transição em pedaços e reduzindo os riscos operacionais de interrupção técnica de forma transparente para os usuários finais.

2.3.2 Ramificação por Abstração (*Branch by Abstraction*)

Quando a necessidade de decomposição envolve componentes e fluxos internos profundos do código que não podem ser isolados unicamente por meio de roteamento de rede periférico, a literatura recomenda o uso do padrão de Ramificação por Abstração (Newman, 2020). De acordo com (Newman, 2020), este método atua de forma interna na base de código através da criação de uma camada de abstração (uma interface) sobre o componente legado que se pretende migrar, forçando com que todo o código cliente interaja estritamente com essa interface.

2.4 Tecnologias Habilitadoras

Para que os benefícios teóricos de uma arquitetura de microsserviços sejam alcançados na prática, é fundamental o emprego de uma base tecnológica capaz de suportar o desenvolvimento isolado, a implantação automatizada e a comunicação distribuída. Neste trabalho, a modernização do sistema apoiou-se no ecossistema Java, na plataforma Docker e no protocolo HTTP.

2.4.1 Linguagem de Programação Java

A linguagem Java consolidou-se como um dos principais padrões para o desenvolvimento de sistemas corporativos (*enterprise*) devido à sua robustez, tipagem estática e à portabilidade garantida pela Máquina Virtual Java (JVM). No contexto de sistemas monolíticos legados, o Java tem sido amplamente utilizado para construir aplicações robustas e centralizadas.

Na transição para arquiteturas distribuídas, o ecossistema Java continuou a ser uma escolha estratégica, especialmente pelo amadurecimento de *frameworks* modernos (como *Spring Boot* e *Quarkus*), que facilitam a criação de aplicações autônomas e autocontidas. A utilização do Java na construção dos novos microsserviços permite o reaproveitamento do conhecimento técnico da equipe e possibilita a extração de módulos do código legado com maior segurança, garantindo que a nova arquitetura mantenha altos níveis de desempenho e confiabilidade no processamento das regras de negócio.

Neste estudo de caso, optou-se pela utilização do framework *Spring Boot*, que simplifica a configuração de microsserviços e provê servidores embutidos para execução imediata.

2.4.2 Plataforma Docker e Contêinerização

A separação de um sistema em múltiplos serviços independentes exige uma infraestrutura que evite conflitos de dependências e reduza o custo computacional de implantação. Nesse cenário, o Docker atua como a principal tecnologia de contêinerização, permitindo o empacotamento de uma aplicação e de todas as suas bibliotecas em uma unidade padronizada e isolada.

Ao contrário das máquinas virtuais tradicionais, que exigem a emulação de um sistema operacional completo e geram grande sobrecarga (*overhead*), os contêineres Docker compartilham o núcleo (*kernel*) do sistema operacional hospedeiro, tornando-os extremamente leves e de inicialização rápida (Tapia *et al.*, 2020). Essa virtualização leve é o que viabiliza a implantação independente citada por (Newman, 2020), pois permite que cada microsserviço seja instanciado, replicado ou desligado de forma ágil, assegurando a paridade entre os ambientes de desenvolvimento, teste e produção.

2.4.3 Protocolo HTTP para Comunicação Distribuída

A transição de uma arquitetura monolítica para microsserviços impõe um desafio fundamental: a redefinição do paradigma de comunicação. Em sistemas centralizados, a interação entre módulos ocorre via chamadas de métodos diretamente na memória, operando com latência quase nula. Contudo, ao fragmentar a aplicação, essa comunicação interna é substituída por chamadas remotas através da rede (Newman, 2020).

Nesse cenário, o Protocolo de Transferência de Hipertexto (HTTP) consolidou-se como a principal escolha para a troca de mensagens. Por ser um protocolo sem estado (*stateless*), universal e leve, o HTTP — frequentemente implementado sob o estilo arquitetural REST (*Representational State Transfer*) — provê a padronização necessária para o diálogo entre serviços independentes. Isso garante que os microsserviços, sejam eles desenvolvidos em Java ou em outras linguagens, exponham suas funcionalidades de maneira uniforme, utilizando métodos verbais bem definidos (como GET, POST, PUT e DELETE) e o tráfego de dados estruturados em formato JSON, assegurando uma integração fluida em todo o ecossistema distribuído.

3 METODOLOGIA

4 DESENVOLVIMENTO

5 DISCUSSÃO DOS RESULTADOS

6 CONCLUSÃO

O objetivo deste documento foi apresentar o uso básico da classe `templufla` para a elaboração de trabalhos acadêmicos da UFLA utilizando $\text{\LaTeX}$. Após edição em $\text{\LaTeX}$, o usuário pode gerar arquivos PDF (Adobe Systems Incorporated, 2004) ou PostScript (Adobe Systems Incorporated, 1999) com grande facilidade.

Em caso de correções ou dúvidas, favor informar.

Contato:

<https://github.com/joaopaulo7/Template-Trabalhos-Academicos-UFLA>

joao.lima10@estudante.ufla.br

REFERÊNCIAS

ADOBE SYSTEMS INCORPORATED. **Postscript Language Reference**. 3. ed. Reading: Addison-Wesley, 1999. Disponível em: http://partners.adobe.com/public/developer/ps/index_specs.html.

ADOBE SYSTEMS INCORPORATED. **PDF Reference: Adobe Portable Document Format, Version 1.6**. 5. ed. San Jose: Adobe Systems Incorporated, 2004. Disponível em: http://partners.adobe.com/public/developer/pdf/index_reference.html.

NEWMAN, S. **Migrando sistemas monolíticos para microserviços**. São Paulo: Novatec, 2020.

TAPIA, F. *et al.* From Monolithic Systems to Microservices: A Comparative Study of Performance. **Applied Sciences**, v. 10, n. 17, p. 5797, ago. 2020. Disponível em: <https://doi.org/10.3390/app10175797>.

ANEXOS

ANEXO A – LOREM IPSUM

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Vivamus semper, libero egestas pellentesque vulputate, velit felis commodo ante, vel bibendum velit turpis eu felis. Donec viverra quam nisi, vel tincidunt enim tristique interdum. Integer tincidunt a lectus vel porttitor. Nulla venenatis vitae enim ut semper. Nunc in sagittis massa, sit amet dapibus quam. Mauris cursus, ligula ac pretium imperdiet, lectus libero egestas mi, quis tristique leo urna nec erat. In vitae dui maximus, imperdiet massa euismod, auctor enim. Morbi urna odio, accumsan quis magna id, fringilla gravida purus. Aenean facilisis est nisi, nec porttitor purus ullamcorper ut. Proin ac risus congue, aliquet elit in, cursus est.

Vivamus lorem diam, molestie ut ultrices at, feugiat quis tortor. Mauris feugiat, augue at molestie malesuada, purus erat sagittis tellus, sit amet posuere lacus nisl non eros. Sed enim justo, sagittis id elementum quis, commodo ut nibh. Aenean mauris odio, efficitur vel purus sit amet, molestie pharetra arcu. Nunc vel eros sodales, aliquam diam eu, rutrum nisi. Morbi non scelerisque diam. Suspendisse sed dapibus mi, ut sagittis nunc. Praesent ornare, est in rutrum dapibus, tortor massa ornare dolor, at ullamcorper metus augue et ipsum. Sed ut nulla in dolor aliquet faucibus. Quisque rhoncus auctor tellus eu lobortis. Proin rhoncus nisi sit amet nibh tempor hendrerit.¹

ANEXO B – LOREM IPSUM PT. 2

Aliquam tempus vehicula risus sit amet consequat. Donec eu mattis lorem. Maecenas tincidunt a massa ut ultricies. Ut id lacus sapien. Suspendisse ac auctor lectus. Maecenas vehicula sagittis metus, eget luctus ipsum imperdiet at. Nullam eu vestibulum leo. Fusce mauris ligula, consequat ut felis a, ornare bibendum quam. Maecenas posuere sem sit amet volutpat interdum. Integer aliquet bibendum luctus. Nulla non viverra eros. Fusce egestas scelerisque augue ac tempor.

¹ Lorem ipsum dolor sit amet, consectetur adipiscing elit. Vivamus semper, libero egestas pellentesque vulputate, velit felis commodo ante, vel bibendum velit turpis eu felis. Donec viverra quam nisi, vel tincidunt enim tristique interdum. Integer tincidunt a lectus vel porttitor.

Aliquam tempus vehicula risus sit amet consequat. Donec eu mattis lorem. Maecenas tincidunt a massa ut ultricies. Ut id lacus sapien. Suspendisse ac auctor lectus. Maecenas vehicula sagittis metus, eget luctus ipsum imperdiet at. Nullam eu vestibulum leo. Fusce mauris ligula, consequat ut felis a, ornare bibendum quam. Maecenas posuere sem sit amet volutpat interdum. Integer aliquet bibendum luctus. Nulla non viverra eros. Fusce egestas scelerisque augue ac tempor.

Aliquam tempus vehicula risus sit amet consequat. Donec eu mattis lorem. Maecenas tincidunt a massa ut ultricies. Ut id lacus sapien. Suspendisse ac auctor lectus. Maecenas vehicula sagittis metus, eget luctus ipsum imperdiet at. Nullam eu vestibulum leo. Fusce mauris ligula, consequat ut felis a, ornare bibendum quam. Maecenas posuere sem sit amet volutpat interdum. Integer aliquet bibendum luctus. Nulla non viverra eros. Fusce egestas scelerisque augue ac tempor.

APÊNDICES

APÊNDICE A – O QUE SÃO APÊNDICES

Um apêndice é um suporte elucidativo e ilustrativo do texto principal. Sua função é agrupar elementos que são úteis à compreensão do texto e que, no entanto, podem ser apresentados à parte sem prejuízo à compreensão. É útil para a apresentação de modelagens, diagramas extensos, listagens de código-fonte de programas e demais elementos que o autor julgar necessário à complementação do tema abordado no texto principal.